REVERSE HARD (500 PTS)

Blackout Protocol

This was a solid 500-pointer by 5h1kh4r. The challenge zip came with a stripped ELF64 executable (`blackout_protocol`) and three forensic artifacts:

- `retained.mem` (16 KB raw memory dump)
- `incident.journal` (custom binary log with `JRN4` magic)
- `transmission.pcap` (290 UDP packets)

Firing up the binary prompts for a 48-byte password. If it passes validation, you get `Authenticated.` ; otherwise, `Blackout.` .

The Gimmick: Intentional Decoys Everywhere

Looking at the binary in IDA/Ghidra, I realized quickly that you couldn't just write a parser based on standard formats. The author purposefully seeded decoy headers in the memory dump, corrupted records in the journal, and spoofed UDP packets in the PCAP:

`$ file blackout_protocol && objdump -d -M intel blackout_protocol | head -n 30`

```
❯ file blackout_protocol
blackout_protocol: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=7bfd5d50c4631fb39ed79ec198004ef8
aa46da0e, for GNU/Linux 6.1.0, stripped

~/SecLeaf writeups/repo_clone/SecLeafCTF/REVERSE/Blackout_Protocol main*
❯ objdump -d -M intel blackout_protocol | head -n 30

blackout_protocol:     file format elf64-x86-64


Disassembly of section .init:

0000000000001000 <.init>:
    1000:       f3 0f 1e fa             endbr64
    1004:       48 83 ec 08             sub    rsp,0x8
    1008:       48 8b 05 c1 2f 00 00    mov    rax,QWORD PTR [rip+0x2fc1]        # 3fd0 <strcmp@plt+0x2f80>
    100f:       48 85 c0                test   rax,rax
    1012:       74 02                   je     1016 <puts@plt-0x1a>
    1014:       ff d0                   call   rax
    1016:       48 83 c4 08             add    rsp,0x8
    101a:       c3                      ret

Disassembly of section .plt:

0000000000001020 <puts@plt-0x10>:
    1020:       ff 35 ca 2f 00 00       push   QWORD PTR [rip+0x2fca]        # 3ff0 <strcmp@plt+0x2fa0>
    1026:       ff 25 cc 2f 00 00       jmp    QWORD PTR [rip+0x2fcc]        # 3ff8 <strcmp@plt+0x2fa8>
    102c:       0f 1f 40 00             nop    DWORD PTR [rax+0x0]

0000000000001030 <puts@plt>:
    1030:       ff 25 ca 2f 00 00       jmp    QWORD PTR [rip+0x2fca]        # 4000 <strcmp@plt+0x2fb0>
    1036:       68 00 00 00 00          push   0x0
    103b:       e9 e0 ff ff ff          jmp    1020 <puts@plt-0x10>

0000000000001040 <strlen@plt>:
    1040:       ff 25 c2 2f 00 00       jmp    QWORD PTR [rip+0x2fc2]        # 4008 <strcmp@plt+0x2fb8>

~/SecLeaf writeups/repo_clone/SecLeafCTF/REVERSE/Blackout_Protocol main*
❯
```

Binary File Triage & Disassembly

Every file feeds cryptographic parameters into the next:

`retained.mem` -> `incident.journal` -> `transmission.pcap` -> S-box/seeds -> Invert 52 rounds -> Flag

Solution Script: `Blackout_Protocol.py` (included in attached solution archive)



GDB Process Memory Layout

Reversing the Pipeline

1. **Memory Dump (`retained.mem`):** A decoy header sits at `0x220`, but the binary explicitly checks offset `0x180` for magic `M16B`. It pulls session tag `0x37` (`'7'`) and XORs two 64-bit words at `0x198` and `0x1a0` to produce constant `r14 = 0xd90ce44398978318`.

2. **Journal (`incident.journal`):** Contains 224 records, but the binary filters strictly on `tag == 0x37`, `idx < 48`, matching FNV-1a checksums, and takes only the *first* occurrence per index. Exactly 48 genuine records survive, populating `arr[48]` and collapsing into chaining hash `H = 0x0e733fa8595bfa24`.

3. **Packet Capture (`transmission.pcap`):** Packets must be processed in raw capture order (sorting by timestamp destroys the PRNG stream). The binary filters for UDP `10.64.0.1:2419 -> 10.64.0.2:31337` starting with `BKP5\x37`. Here was the biggest trap: 72 forged packets pass a rolling 16-bit checksum, but fail a second 32-bit check (`chk32`) keyed by `arr[(7*slot + 3) % 48]` from the journal. Filtering strictly by `chk32` leaves exactly 32 genuine packets, which decrypt via SplitMix64 into a 384-byte keystream `K`.

The 52-Round Cipher

Once all parameters are extracted, the binary hashes them into a master seed: `seed = mix64(tag ^ r14 ^ H ^ K[0:8] ^ K[73:81]) = 0x98b5f1c61c260485` This seed builds an S-box (via Fisher-Yates shuffle with `xorshift64*`) and runs 52 rounds of a Substitution-Permutation Network (SPN) on the 48-byte input: 1. S-box substitution: `S[j] = T[S[j] ^ f(round, j)]` 2. Affine byte permutation: $S'[(7round + 11 + 17k) \% 48] = S[k]$

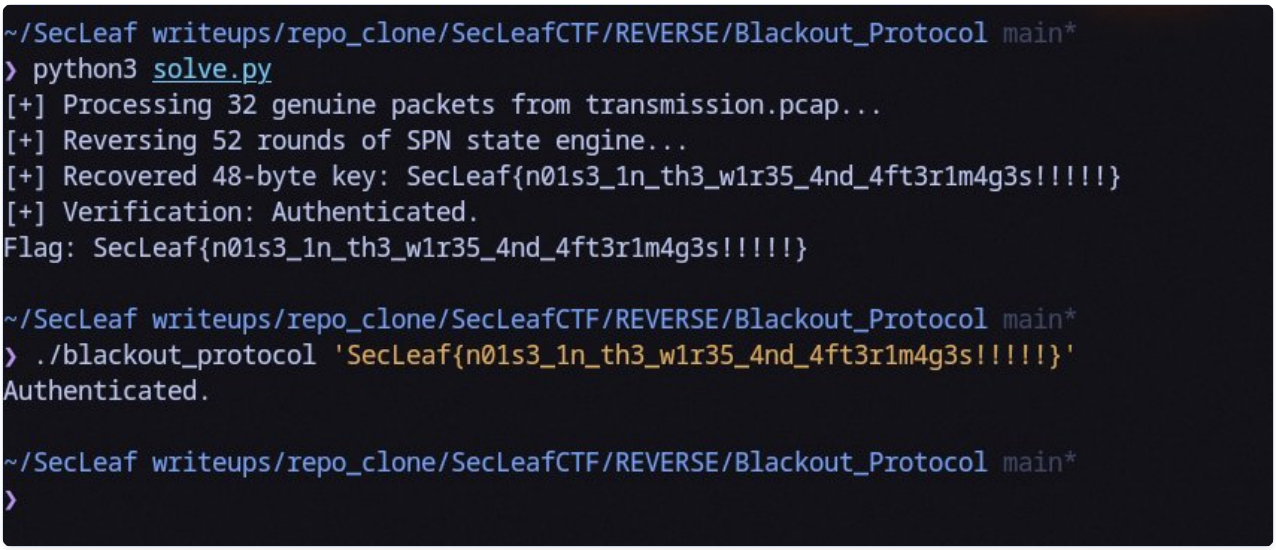
Since each round is strictly bijective, reversing the cipher was straightforward: start at the target ciphertext in the binary, invert the permutation, and apply the inverse S-box for rounds 51 down to 0.

Solving

Put together the full inversion script: **Solution Script:** `Blackout_Protocol.py` (included in attached solution archive)

Ran the script to recover the 48-byte password and validated it against the binary:

```
$ python3 solve.py $ ./blackout_protocol 'SecLeaf{n01s3_1n_th3_w1r35_4nd_4ft3r1m4g3s!!!!!}'
```



Solver Execution & Binary Flag Acceptance

Solution Script: Blackout_Protocol.py (included in attached solution archive)

Flag:

Solution Script: Blackout_Protocol.py (included in attached solution archive)

RECOVERED FLAG **SecLeaf{n01s3_1n_th3_w1r35_4nd_4ft3r1m4g3s!!!!!}**